Phase 2 (Estimation Engine + E-Signature) — Implementation Plan

Date: 2026-07-09 **Depends on:** Phase 1 (CRM & Project Management) — merged to `main` at `185d263`. **Scope source:** `docs/02-functional-requirements.md` Section 4, `docs/09-roadmap-implementation-plan.md` Phase 2, `docs/04-database-schema.md` Sections 4 (`change_orders`), 5–6, `docs/05-api-specification.md` Sections 4–5, `docs/03-technical-architecture.md` Sections 4, 6–7, `docs/07-security-compliance.md` Section 6.

Phase 2 Scope (from the Roadmap, verbatim)

- Cost Catalog, Markup Profiles (with parent/child override inheritance).
- Estimate creation, line items, server-side calculation pipeline (fixed-point decimal, fixed order of operations).
- PDF export as an async job.
- E-signature capture flow for Estimate approval (`esignatures` table).
- Change Order creation + e-signature approval on active Projects (reuses the e-signature capability built for Estimates).
- Historical immutability / snapshotting on approval.
- `ESTIMATE_APPROVED` event published (consumed starting in Phase 3).
- **Exit criteria: this is the MVP launch bar.** Users/Company + CRM + Project Management + Estimation Engine (including e-signature) are feature-complete, tested, and deployed to production.

Caveat on "deployed to production": this plan covers the feature-complete, tested build (code, migrations, tests, CI) — the literal roadmap exit criterion also names production deployment to the developer's self-hosted Proxmox infrastructure (PRD Section 7, NFRs Section 6). That's a manual infrastructure/ops step outside what an implementation plan executed by coding agents can perform (no access to the physical host). This plan's own exit criteria (below) are scoped to what's actually buildable: feature-complete, tested, CI-green, and mergeable — matching how Phase 0/1's plans handled their own exit criteria.

Explicitly Out of Scope for Phase 2 (deferred, not forgotten)

- **Subcontractors, compliance documents, compliance dashboard.** `docs/04-database-schema.md` Section 6 groups `subcontractors/compliance_documents/subcontractor_assignments` under "Cross-Cutting" alongside `esignatures`, but `docs/09-roadmap-implementation-plan.md` explicitly places all of Compliance Tracking in **Phase 3**. Only `esignatures` itself is built in Phase 2 — it's the one

table from that section actually required by this phase's own scope (Estimate/Change Order approval).

- **Billing/invoicing, `ESTIMATE_APPROVED`'s actual consumer.** This phase *publishes* `ESTIMATE_APPROVED` (roadmap's own explicit instruction: "consumed starting in Phase 3") but registers no handler for it — same "publish now, wire the consumer later" pattern Phase 1 established for `LEAD_WON` during its own Task 1.5 (a real, callable `publish()` call with zero handlers is a no-op, not a TODO).
- **QuickBooks/FreshBooks, Stripe subscription billing, profitability reporting.** Phase 3–4.
- **A logo-upload / true visual branding system for PDF proposals.** "Branded PDF proposal" (US-4.4) is satisfied with a clean, consistent, company-name-labeled template — no `companies.logo_path` column exists in the schema doc, and adding one plus an upload flow is its own small feature, not required by any Phase 2 user story's acceptance criteria. Revisit if a future phase calls for it explicitly.
- **Anonymous/magic-link e-signature flow for a Client who isn't a platform user.** See design decision #3 below — Client is an authenticated `company_users` role in this schema, not an anonymous external signer.
- **Refresh-token rotation, MFA, offline/mobile PWA support** (carried over from Phase 0/1's deferred lists, still not triggered).

Inherited Invariants from Phase 0/1 (apply to every new table/route — do not rediscover these bugs)

These are load-bearing lessons from Phase 0's 10 and Phase 1's 8 design decisions (`docs/superpowers/plans/2026-07-07-phase-0-foundation.md`, `docs/superpowers/plans/2026-07-08-phase-1-crm-project-management.md`). Every task below assumes them; call them out explicitly in review if a task's diff doesn't follow one.

1. **Every RLS policy cast must be `NULLIF(current_setting('app.x', true), '')::uuid`, never a bare cast.** A bare cast raises an unhandled error once a pooled connection has ever seen that GUC set.
2. **Every UPDATE policy needs an explicit `WITH CHECK`, not just `USING`.**
3. **Route handlers reuse `current.session` from `get_current_user`/`require_role`; never open a new session, never call `session.commit()` inline.** The dependency owns the transaction and commits once, after the handler returns. **This invariant does NOT extend to the Dramatiq worker process (design decision #5 below)** — a background job runs in a separate OS process with no HTTP request/response cycle to hang a `yield`-based dependency off

of; it manages its own session and commit explicitly. Don't try to force worker code through `get_current_user`.

4. **Migrations run as the `postgres` owner role; the app connects as `app_user`, which is subject to RLS.** Never grant `app_user` table ownership. **Never use the owner/migrations connection from application (request or worker) code as a way to "see more" than RLS would otherwise allow** — this is directly relevant to design decision #1 below (Cost Catalog inheritance), which resolves a real upward-visibility need via a new bidirectional RLS policy specifically because reaching for the owner connection to bypass RLS in ordinary business logic would break Phase 0's entire "RLS is authoritative" security model.
5. **Regression Testing Policy (unchanged):** every task's verification runs the FULL test suite (`pytest -v`, no path filter). A task is not done until the full suite is green.
6. **Every task goes through implementer → independent spec-compliance review → independent code-quality review**, each re-verifying claims against the live database, not trusting the prior report. Every task from Phase 0's #5 onward and the majority of Phase 1's tasks found at least one real bug this way — this discipline is not optional process theater.
7. **RBAC role checks reuse `require_role(*roles)`** (`app/core/deps.py`) as-is.
8. **`404` = "doesn't exist or isn't visible to you" (RLS-backed, existence-indistinguishable); `403` = "visible, but this role/field/ownership rule blocks the action."** Established across every Phase 1 router (`_get_lead_or_404`, `_get_project_or_404`, `_get_task_or_404`) — apply the identical split to `_get_estimate_or_404`, `_get_change_order_or_404`, etc.
9. **All monetary values use `Decimal`, never `float`**, from the Postgres `NUMERIC` column type through Pydantic schemas through the calculation engine itself — this is a Phase 2-specific invariant, not inherited, but stated here because it applies to literally every new model/schema this phase touches and a single `float` slipping in anywhere breaks the whole guarantee.

New Critical Design Decisions for Phase 2

1. **Cost Catalog parent/child inheritance requires a NEW, bidirectional RLS policy on `cost_catalog_items` — not an application-layer tenant-context walk.** This is the trickiest design problem in this phase; read carefully.

   US-4.6 requires: "child branches able to override inherited values… a child branch's local override takes precedence over the parent's catalog entry for that item; the parent catalog itself is unaffected."

This means a **child-branch session must be able to READ its parent's (and grandparent's, etc.) catalog items** — visibility flowing *upward* — which every existing RLS policy in this codebase (`companies`, `leads`, `projects`, `phases`, `tasks`, `documents`, `daily_logs`) explicitly does NOT support: `get_all_descendant_ids(current_tenant)` only returns the active tenant plus its own descendants, never its ancestors. A parent sees its children's data; a child cannot see its parent's.

Two ways to get upward visibility were considered and rejected:

- **Reach for the `postgres` owner connection from application code** to read ancestor rows, bypassing RLS. Rejected: this breaks Phase 0's foundational security model (owner access is reserved for migrations only, per Inherited Invariant #4 above) and would need to be re-derived/re-audited for every future table that ever needs similar inheritance — a precedent this codebase should not set.
- **Temporarily re-`SET LOCAL app.current_tenant` to each ancestor company_id in turn, within the same transaction, to query each one, then restore the original tenant context before returning.** Rejected as too dangerous: `set_current_tenant()` (`app/db.py`, Phase 0) is a real, transaction-scoped GUC mutation — if a resolver forgot to restore the original tenant context (or raised partway through, skipping the restore), every subsequent query in that same request/transaction would silently run in the wrong tenant's context. This is exactly the class of bug Phase 0/1's RLS discipline exists to prevent, and manufacturing it inside a "read the catalog" helper is not worth the risk for a read-only convenience feature.

Decision: extend `cost_catalog_items`' RLS policy to grant visibility in BOTH directions — same downward (parent-sees-descendants) visibility every other table has, PLUS a new upward (child-sees-ancestors) grant, via a new SQL function mirroring the existing one:

```
CREATE OR REPLACE FUNCTION get_all_ancestor_ids(company_uuid
UUID)
RETURNS TABLE (ancestor_id UUID) AS $$
    WITH RECURSIVE ancestor_tree AS (
        SELECT id, parent_id FROM companies WHERE id =
company_uuid
        UNION ALL
        SELECT c.id, c.parent_id FROM companies c INNER JOIN
ancestor_tree at ON c.id = at.parent_id
    )
```

```
    SELECT id FROM ancestor_tree;
$$ LANGUAGE sql STABLE;

CREATE POLICY tenant_isolation_policy ON cost_catalog_items
    USING (
        company_id IN (SELECT id FROM
get_all_descendant_ids
(NULLIF(current_setting('app.current_tenant', true), '')::uuid))
        OR company_id IN (SELECT id FROM
get_all_ancestor_ids(NULLIF(current_setting('app.current_tenant',
true), '')::uuid))
    )
    WITH CHECK (
        company_id IN (SELECT id FROM
get_all_descendant_ids
(NULLIF(current_setting('app.current_tenant', true), '')::uuid))
    );
```

Note `WITH CHECK` is intentionally NOT bidirectional — a session can only ever *write* rows scoped to its own descendant set (itself or a branch it administers), never fabricate a write into an ancestor's row it merely has read visibility into. This is the same "read broader than you can write" shape as nothing else in this codebase yet, worth calling out explicitly in the migration's own comments so a future reader doesn't "fix" the asymmetry as an apparent bug.

This makes an ordinary RLS-scoped `SELECT * FROM cost_catalog_items` return every visible row (the caller's own overrides AND every ancestor's shared items) with **zero manual tenant-context juggling** — the database does the visibility work, exactly matching this project's established "RLS is authoritative" philosophy. The only application-layer work left is **dedup-and-prefer-closest**: when the same conceptual item exists at multiple levels of the hierarchy (an override chain via `parent_catalog_item_id`), the resolver keeps the row belonging to the company closest to the caller's own position in the tree and discards the rest. This resolution logic lives in `app/services/catalog_resolution.py` (Task 2.4).

`markup_profiles` has NO `parent_profile_id` column in the schema doc — it is a plain, flat, per-company resource, not inheritable/overridable the way `cost_catalog_items` is. Do not add inheritance logic for markup profiles; that would be scope creep beyond what US-4.6 and the schema actually specify.

2. **A real async task queue (Dramatiq + Redis) is introduced for the first time in Phase 2 — this is the first phase where standing one up is actually justified.** Technical Architecture Section 7 names "Celery or Dramatiq + Redis" for PDF generation,

QuickBooks sync, and notification delivery, but explicitly ties it to "Phase 2+" — Phase 1's own design decision #2 deliberately deferred it, reasoning that `LEAD_WON`'s single in-process, same-transaction consumer didn't need it. PDF export (US-4.4, this phase) is different in kind: NFRs Section 1 requires it complete asynchronously within 30s, and API Specification Section 5 documents `POST /estimates/{id}/export` returning `202 Accepted` — a real out-of-request-cycle job, not something that can be squeezed into the same transaction the way `LEAD_WON`'s Project-drafting was.

Choosing Dramatiq over Celery: fewer moving parts (no separate result backend needed for a fire-and-forget job whose "result" is just a file on disk plus a status flag on the `estimates` row), simpler configuration for a solo-developer self-hosted deployment, and native `async`/`sync` interop that fits this codebase's existing async-first FastAPI/SQLAlchemy stack more naturally than Celery's traditionally-sync worker model. `redis_url` already exists in `Settings` (added in Phase 0, unused until now) — Phase 2 is the first phase to actually wire it to something real.

New `docker-compose.yml` service: `worker` — same backend image (`build: ./backend`), different `command` (`dramatiq app.tasks.estimate_pdf`), same `env_file: .env`, `depends_on: {postgres: service_healthy, redis: service_healthy}`. No published ports (not HTTP-reachable).

The worker is a separate OS process — it does NOT get `current.session` from the enqueuing request. It opens its own `AsyncSession`, calls `set_current_tenant()`/`set_current_user()` (both already exist in `app/db.py` from Phase 0, just not currently called outside `get_current_user`) explicitly for its own transaction, does its work, commits, and closes — mirroring `get_current_user`'s pattern but without the FastAPI dependency-injection lifecycle wrapping it. If the job fails, the request that enqueued it has already returned `202` — there is no "roll back the original request" story here, only "the job failed, and `estimates.pdf_status` reflects `'failed'`." This is a genuinely different transactional/consistency model than `LEAD_WON`'s in-process, single-transaction guarantee (Phase 1 design decision #2's whole point), and this difference must be documented, not silently glossed over as "just another event."

3. **E-signature capture is an authenticated in-app action by a `client`-role user, not an anonymous emailed-link flow.** US-4.5's literal phrasing ("As a Client, I can review an emailed Estimate and approve it with an e-signature") could be misread as implying a magic-link, no-login signing flow for an external party who isn't a platform user at all. But `company_users.role` (per Database Schema

Section 2) has a `CHECK` constraint including `'client'` as a first-class role, and the RBAC matrix (Security & Compliance Section 2) explicitly grants `Client: Approve/reject own estimate (e-sign)` — meaning Client is invited into the platform via the exact same `Invitation` flow every other role uses (built in Phase 0), logs in, and approves while authenticated. "Emailed" in US-4.5 describes the notification (an email alerting them an Estimate is ready for review), not the signing mechanism itself. This decision keeps e-signature capture inside the existing auth/RBAC system with zero new infrastructure (no magic-link tokens, no anonymous-signer session type) — the signer's identity, IP, and timestamp are all captured from their normal authenticated session.

"Own estimate" scoping: `estimates` has no `client_id`/signer-linkage column, and `client` role visibility is company-scoped (via ordinary RLS), not project- or estimate-specific. A `client`-role user can approve/reject any `estimate` with `status='sent'` that RLS already makes visible to them (i.e., anything in their own tenant) — there is no per-estimate assignment concept for clients in this schema, matching how `client` already gets blanket (not project-by-project-granted) sanitized read access to Projects in Phase 1.

4. **Historical immutability (`estimates.is_snapshotted`) is enforced at the APPLICATION layer, not the DB-grant layer — a deliberate, documented deviation from Phase 1's "prefer DB-level enforcement" instinct.** Phase 1 design decision #6 REVOKEd UPDATE/DELETE unconditionally at the grant level for tables that are *always* immutable (`communication_logs`, `daily_logs`, `documents`). `estimate_line_items`/`estimates.total`/`estimates.subtotal` are different: they're **mutable while `is_snapshotted = false`** (a PM actively building out an Estimate needs to add/edit/remove line items and recalculate freely) and **become immutable only once `is_snapshotted` flips to `true`** (on approval). A blanket `REVOKE UPDATE` would break the normal editing workflow entirely; a GRANT can't be conditioned on another column's runtime value. This is therefore enforced the same way Phase 1 enforced `PATCH /tasks/{id}`'s field_crew field-restriction (an explicit application-layer check before any mutation, tested directly): `PUT /estimates/{id}/lines` and `POST /estimates/{id}/calculate` both check `estimate.is_snapshotted` first and return `409 Conflict` if true, before touching any row. Once `is_snapshotted = true`, `estimate_line_items.unit_rate_snapshot/line_total` and the parent `estimates.subtotal/total` are never written again by any code path.

5. **PDF export status/location is tracked via new columns on `estimates` itself, not a separate export-history table.** The schema doc's `estimates` table has no PDF-tracking columns at all — a real gap,

the same category as Phase 1's `PATCH /projects/{id}` design decision #3 (necessary extension beyond the documented schema, not a deviation from intent). Add `pdf_status VARCHAR(20) NOT NULL DEFAULT 'not_requested' CHECK (pdf_status IN ('not_requested','pending','ready','failed'))`, `pdf_storage_path TEXT` (nullable, same `STORAGE_ROOT`-relative convention `document_storage.py` established in Phase 1, reused directly — see Task 2.13), and `pdf_generated_at TIMESTAMPTZ` (nullable). A dedicated history table is unnecessary: only the most recent export matters practically (re-exporting after a line-item edit produces a new PDF from current state, previous exports aren't a retained artifact the way Document versions are), and `documents`' project-scoped versioning model doesn't fit here anyway (an Estimate can exist against a bare Lead with no Project yet, per US-4.1).

6. **`change_orders` is finally built in Phase 2**, unblocked now that `esignatures` exists (Phase 1 explicitly deferred it for exactly this reason — see that plan's top-of-doc note). Reuses the exact same shared e-signature capture service as Estimates (`app/services/esignature.py`, Task 2.10) — `POST /change-orders/{id}/send-for-signature` and the client-facing approval action call the identical function `POST /estimates/{id}/send-for-signature` does, parameterized by `document_type` (`'estimate'` vs `'change_order'`), matching the roadmap's own explicit framing ("reuses the e-signature capability built for Estimates").

7. **The Change-Order-blocks-Project-completion business rule, deferred by Phase 1's `project_transitions.py` with an explicit "do not forget" comment, is implemented in this phase.** Functional Requirements Section 3: "A Project cannot move to Completed while it has open (non-approved) Change Orders." Phase 1's own module docstring in `app/services/project_transitions.py` (lines 39–52) already documents exactly what's needed: an additional application-layer check — querying for any `change_orders` row with `status != 'approved'` against the target project — layered on top of (not replacing) the existing table-driven transition check, applied to BOTH `active → completed` and `suspended → completed` (not just one), since keying it off "coming from suspended" specifically would silently miss the more common `active → completed` path.

8. **`ESTIMATE_APPROVED` reuses Phase 1's existing `app/core/events.py` dispatcher as-is** — no new event-bus machinery. `publish("ESTIMATE_APPROVED", session=current.session, estimate_id=..., project_id=..., approved_total=..., company_id=...)` is called from the approval endpoint; `register_event_handlers()` (`app/core/event_handlers.py`, Phase 1) is the one place a Phase 3 handler will later be added. Same "publish

now, wire the consumer later" split Phase 1 itself used for `LEAD_WON` between Tasks 1.5 and 1.18.

Regression Testing Policy (unchanged from Phase 0/1)

Every task's verification step runs `cd backend && pytest -v` (no path filter) after its own new/changed test file passes in isolation. A task is not done until the full suite is green. Any new migration additionally requires a full tenant-isolation adversarial pass for its new tables before being considered complete (see Tasks 2.5, 2.17, 2.23).

File Structure (additions to Phase 0/1's layout)

```
backend/
  migrations/versions/
    0005_cost_catalog_and_markup_schema.py
    0006_esignatures_schema.py
    0007_estimates_schema.py
    0008_change_orders_schema.py
  app/
    config.py                          # + storage_root reuse note, +
dramatiq broker settings
    models/
      markup_profile.py
      cost_catalog_item.py
      esignature.py
      estimate.py
      estimate_line_item.py
      change_order.py
    schemas/
      markup_profile.py
      cost_catalog_item.py
      esignature.py
      estimate.py
      estimate_line_item.py
      change_order.py
    routers/
      catalogs.py                      # /catalogs/items, /markup-
profiles
      estimates.py
      change_orders.py                 # or folded into projects.py — see
Task 2.21
    services/
      catalog_resolution.py            # inheritance dedup-and-prefer-
closest (design decision #1)
      estimate_calculation.py          # fixed-order Decimal pipeline
(Task 2.12)
      esignature.py                    # shared capture service (design
decision #3/#6)
      pdf_export.py                    # WeasyPrint/Jinja2 rendering
(Task 2.13)
```

```
    tasks/
      __init__.py
      broker.py                       # Dramatiq broker setup (design
decision #2)
      estimate_pdf.py                 # the actual Dramatiq actor
    templates/
      estimate_pdf.html.jinja         # PDF template
  tests/
    test_cost_catalog.py
    test_cost_catalog_inheritance.py
    test_markup_profiles.py
    test_estimates.py
    test_estimate_calculation.py
    test_estimate_snapshotting.py
    test_esignatures.py
    test_estimate_pdf_export.py
    test_change_orders.py
    test_project_completion_blocked_by_change_orders.py
    test_tenant_isolation_phase2.py   # cross-tenant regression for
every new table
scripts/
  e2e_smoke_test.py                   # extended with the Estimate-to-
Approval flow
docker-compose.yml                    # + worker service (design
decision #2)
```

Task 2.1: MarkupProfile & CostCatalogItem Models

Files: Create `backend/app/models/markup_profile.py`, `backend/app/models/cost_catalog_item.py`. Modify `backend/app/models/__init__.py`.

- `MarkupProfile`: `id`, `company_id` (FK, not null), `name`, `overhead_pct` (`Numeric(5,2)`, not null, default 0), `profit_pct` (`Numeric(5,2)`, not null, default 0). No `created_at`/`updated_at` in the schema doc — match it exactly, don't add timestamps the doc doesn't specify.
- `CostCatalogItem`: `id`, `company_id` (FK, not null), `parent_catalog_item_id` (FK to self, nullable — "links a branch override to its parent's item"), `category`, `name`, `unit`, `unit_rate` (`Numeric(12,2)`, not null), `updated_at`.
- Follow the exact SQLAlchemy model conventions established across Phase 0/1 (`UUIDPKMixin`, `TimestampMixin`/`UpdatedAtMixin` only where the schema doc actually specifies a matching column — `MarkupProfile` gets neither mixin, `CostCatalogItem` gets `updated_at` only, no `created_at`).
- Verify: full suite green (no behavior change yet).
- Commit: `feat: add MarkupProfile and CostCatalogItem models`

Task 2.2: Cost Catalog & Markup Migration (0005)

Files: Create `backend/migrations/versions/0005_cost_catalog_and_markup_schema.py`.

- `down_revision = "0004"`.
- `CREATE TABLE markup_profiles`, `cost_catalog_items` exactly per Database Schema Section 5.
- `CREATE OR REPLACE FUNCTION get_all_ancestor_ids(company_uuid UUID)` per design decision #1 — a new recursive function, sibling to Phase 0's existing `get_all_descendant_ids`, walking `parent_id` upward instead of downward.
- `markup_profiles` RLS: standard `FOR ALL tenant_isolation` policy, guarded-cast `USING/WITH CHECK` pattern (Inherited Invariant #1/#2), same shape as every Phase 1 table — no inheritance, plain company-scoped visibility.
- `cost_catalog_items` RLS: the NEW bidirectional policy from design decision #1 — asymmetric `USING` (descendants OR ancestors) vs `WITH CHECK` (descendants only). Comment this asymmetry explicitly in the migration file itself, not just the plan doc, so a future reader of the migration alone understands why it looks different from every other table's policy.
- Verify by hand against live Postgres, same discipline as every prior migration task: apply the migration, confirm `\d+ cost_catalog_items` shows RLS enabled with the two-clause policy, and empirically confirm (via raw `asyncpg`, not just reading the SQL) that a session scoped to a CHILD company can `SELECT` a row owned by its PARENT, and vice versa for the descendant direction — this is exactly the kind of policy shape that's easy to get subtly backwards, verify it does what's intended before moving on.
- Run the full suite (still green — nothing references these tables yet).
- Commit: `feat: add Cost Catalog and Markup Profile schema migration`

Task 2.3: MarkupProfile & CostCatalogItem Schemas

Files: Create `backend/app/schemas/markup_profile.py`, `backend/app/schemas/cost_catalog_item.py`.

- `MarkupProfileCreateRequest`: `name`, `overhead_pct` (`Decimal`, default `0`), `profit_pct` (`Decimal`, default `0`) — use Pydantic's `Decimal` type, never `float` (Inherited Invariant #9). `MarkupProfileResponse`: full model, `ConfigDict(from_attributes=True)`.
- `CostCatalogItemCreateRequest`: `category`, `name`, `unit`, `unit_rate` (`Decimal`). No `parent_catalog_item_id` in the create request — see

Task 2.4 for why creating an *override* is a distinct operation from creating a brand-new catalog item, not a field on the same schema.

- `CostCatalogItemResponse`: full model + a computed `is_override: bool` (`parent_catalog_item_id is not None`) for frontend clarity — cheap to add now, avoids the frontend needing to infer it from a nullable FK.
- Verify: full suite green.
- Commit: `feat: add MarkupProfile and CostCatalogItem Pydantic schemas`

Task 2.4: Cost Catalog Inheritance Resolution Service

Files: Create `backend/app/services/catalog_resolution.py`. Create `backend/tests/test_cost_catalog_inheritance.py`.

- `resolve_visible_catalog_items(session, active_company_id) -> list[CostCatalogItem]`: runs an ordinary RLS-scoped `SELECT * FROM cost_catalog_items` (the bidirectional policy from Task 2.2 already returns every visible row — both the caller's own rows and every ancestor's shared items) and then applies dedup-and-prefer-closest: group rows by "conceptual item identity" (an item and its override chain, walked via `parent_catalog_item_id` — the root of a chain, or the row itself if it has no parent link, is the identity key) and keep only the row belonging to the company closest to `active_company_id` in the tree (0 hops = the caller's own override, 1 hop = immediate parent's, etc. — compute hop-distance by walking `parent_id` from `active_company_id` upward and noting each ancestor's position).
- This function does NOT itself call `set_current_tenant()` or touch tenant context in any way — it trusts the session's ALREADY-established RLS context (set once, normally, by `get_current_user`) and relies entirely on the new bidirectional policy to have already scoped the rows correctly. Say this explicitly in the function's docstring: this is precisely what makes it safe (Inherited Invariant #4/design decision #1's whole point) — no manual context-switching anywhere in application code.
- Tests: a plain company with no parent sees only its own catalog; a child branch with no overrides of its own sees its parent's full catalog; a child branch with ONE override for an item sees that item as its own override and every OTHER item as the parent's; a grandchild branch overriding an item its PARENT (not grandparent) already overrode sees its own (closest) version, not the grandparent's original; the parent's own view of its catalog is completely unaffected by any child's override (US-4.6's explicit "the parent catalog itself is unaffected" acceptance criterion) — verify by checking the parent's resolved list still shows the parent's own original row, not a child's override leaking upward.
- Verify: full suite green.

- Commit: `feat: add cost catalog inheritance resolution service`

Task 2.5: `POST/GET /catalogs/items`, `POST /markup-profiles`

Files: Create `backend/app/routers/catalogs.py`. Modify `backend/app/main.py`. Create `backend/tests/test_cost_catalog.py`, `backend/tests/test_markup_profiles.py`.

- `POST /catalogs/items`: `require_role("admin", "project_manager")` per the RBAC matrix (Estimation = Admin + PM full CRUD). Two distinct creation shapes, both hitting this one route based on payload: a brand-new catalog item (`parent_catalog_item_id` absent) vs. an override of a visible ancestor item (client supplies the ancestor's `id` as `parent_catalog_item_id` in a *separate*, explicit query param or route — decide based on what reads more cleanly, e.g. `POST /catalogs/items/{parent_id}/override` as a distinct route vs. accepting `parent_catalog_item_id` in the body; use judgment, but whichever is chosen, validate the referenced `parent_catalog_item_id` is actually visible to the caller via `resolve_visible_catalog_items` first — a caller can't override an item they can't see, which the bidirectional RLS policy alone doesn't prevent them from *attempting* since `WITH CHECK` only constrains the row's own `company_id`, not the FK target's visibility).
- `GET /catalogs/items`: paginated (reuse `app/core/pagination.py`, Phase 1's established pattern), returns `resolve_visible_catalog_items`'s output — NOT a raw RLS-scoped query, since raw would show both the parent's original AND the child's override for the same item, contradicting "a child branch's local override takes precedence." Category filter and search (per API spec) apply AFTER resolution, on the deduped list.
- `POST /markup-profiles`: `require_role("admin", "project_manager")`, plain company-scoped create, no inheritance logic (design decision #1's closing note).
- `GET /markup-profiles`: paginated, plain company-scoped list.
- Tests: create catalog item, create override (parent item correctly identified, unaffected), list shows resolved (deduped, override-preferred) view, category/search filters apply post-resolution, cross-tenant 404 on direct catalog-item ID access from an unrelated tenant (not a parent/child — genuinely unrelated), markup profile create/list, non-admin/PM roles blocked.
- Verify: full suite green.
- Commit: `feat: add Cost Catalog and Markup Profile routes with inheritance resolution`

Task 2.6: Cost Catalog Tenant-Isolation Regression Tests

Files: Start `backend/tests/test_tenant_isolation_phase2.py`.

- Mirror Phase 1's Task 1.8/1.17 rigor, adapted for `cost_catalog_items`' NEW bidirectional policy specifically: (a) a genuinely unrelated tenant (no ancestor/descendant relationship at all) sees nothing of another tenant's catalog, in either direction — direct-ID access blocked (404), header-spoofing blocked (403); (b) an RLS-disable/re-enable regression test (Phase 0 Task 16's pattern) proving the POLICY itself, not `resolve_visible_catalog_items`' application-layer dedup, is what blocks unrelated-tenant access — connect as `app_user` directly, confirm invisibility, disable RLS, confirm the row becomes visible, re-enable, confirm invisibility returns; (c) explicitly verify the SIBLING case (two children of the same parent) — sibling B's catalog item is invisible to sibling A even though both are "descendants of the same ancestor," since the bidirectional policy's ancestor-visibility grant is scoped to the caller's OWN ancestor chain, not lateral relatives.
- Verify: full suite green, run twice.
- Commit: `test: add tenant-isolation regression coverage for Cost Catalog (bidirectional RLS)`

Task 2.7: Estimate & EstimateLineItem Models

Files: Create `backend/app/models/estimate.py`, `backend/app/models/estimate_line_item.py`. Modify `backend/app/models/__init__.py`.

- `Estimate`: `id`, `company_id` (FK), `project_id` (FK, nullable), `lead_id` (FK, nullable), `markup_profile_id` (FK, not null), `status` (string, default `"draft"`), `subtotal`/`total` (`Numeric(12,2)`, nullable — unset until first calculation), `is_snapshotted` (bool, not null, default `false`), `esignature_id` (FK, nullable), `created_at`, `updated_at`. Plus design decision #5's three new PDF-tracking columns: `pdf_status` (string, not null, default `"not_requested"`), `pdf_storage_path` (nullable text), `pdf_generated_at` (nullable timestamptz).
- `EstimateLineItem`: `id`, `estimate_id` (FK, cascade), `company_id` (FK), `cost_catalog_item_id` (FK, not null), `quantity` (`Numeric(12,2)`, not null), `unit_rate_snapshot` (`Numeric(12,2)`, not null — copied at add-time, per schema doc's own Section 9 note: "intentionally separate columns… this is what implements the historical-immutability rule"), `line_total` (`Numeric(12,2)`, not null). No `created_at`/`updated_at` — match the schema doc exactly.
- Verify: full suite green.
- Commit: `feat: add Estimate and EstimateLineItem models`

Task 2.8: Estimates Migration (0007 — see re-sequencing note below)

Files: Create `backend/migrations/versions/0007_estimates_schema.py`.

- `down_revision = "0005"`. Note this migration's FK dependency ordering: `estimates.esignature_id` references `esignatures(id)`, but `esignatures` doesn't exist yet — per Database Schema's own top-of-doc note ("actual Alembic migrations must sequence CREATE TABLE statements by foreign-key dependency, not by this document's section order"), **`esignatures` must be created in an EARLIER migration than this one.** Re-sequence: `esignatures` becomes migration `0006`, `estimates/estimate_line_items` becomes `0007`. Update the File Structure section above and every reference in this plan doc accordingly if this re-sequencing changes numbering — cross-check before starting Task 2.9's migration file name.

  (This note is itself a planning correction, not an oversight to preserve — the File Structure list above already reflects the corrected order: `0005` cost catalog/markup, `0006` esignatures, `0007` estimates. Follow that order, not this task's own literal number in isolation.)

- `CREATE TABLE estimates, estimate_line_items` per the schema doc plus design decision #5's three PDF columns (not in the schema doc — a documented, deliberate extension, same category as Phase 1's `PATCH /projects/{id}`).

- Standard `tenant_isolation` policy (guarded-cast, `WITH CHECK`) on both tables — no inheritance concern here, estimates aren't shared across the company hierarchy the way catalog items are.

- No `REVOKE` on `estimate_line_items` at the grant level (design decision #4 — immutability is conditional on `is_snapshotted`, enforced in application code, not at the grant level).

- Verify by hand against live Postgres per the established discipline.

- Run the full suite (still green).

- Commit: `feat: add Estimates schema migration`

Task 2.9: Estimate & EstimateLineItem Schemas

Files: Create `backend/app/schemas/estimate.py`, `backend/app/schemas/estimate_line_item.py`.

- `EstimateCreateRequest`: `project_id` (optional), `lead_id` (optional), `markup_profile_id` — validate in the router (not the schema, since it

needs a DB lookup) that exactly one of `project_id`/`lead_id` is meaningfully associated per US-4.1 ("against a Lead or Project"); both being absent or an unrelated combination should be rejected. Use judgment on exact validation shape (both nullable at the schema level is fine — Pydantic can't cross-validate against the DB — but the router must enforce "at least one of the two, and if a lead_id is given its status must be 'estimating' or later" per Functional Requirements Section 2's "Lead status 'Estimating' is the trigger point that allows an Estimate to be created against it").

- `EstimateResponse`: full model including the PDF-tracking fields (`pdf_status`, `pdf_storage_path`, `pdf_generated_at`) so the frontend can poll this exact route for export readiness (design decision #5).
- `EstimateLineItemInput` (used inside the batch-replace request body, not a standalone create route): `cost_catalog_item_id`, `quantity` (`Decimal`).
- `EstimateLineItemsReplaceRequest`: `items: list[EstimateLineItemInput]` — matches `PUT /estimates/{id}/lines`'s documented "batch replace line items" shape (API spec Section 5).
- `EstimateLineItemResponse`: full model.
- Verify: full suite green.
- Commit: `feat: add Estimate and EstimateLineItem Pydantic schemas`

Task 2.10: `POST /estimates`, `GET /estimates`, `GET /estimates/{id}`

Files: Create `backend/app/routers/estimates.py`. Modify `backend/app/main.py`. Create `backend/tests/test_estimates.py`.

- `POST /estimates`: `require_role("admin", "project_manager")`. Validates the lead/project association rule from Task 2.9. Starts `status="draft"`, zero line items, `subtotal`/`total` unset (`NULL`), `is_snapshotted=false`. Audit log entry (`estimate.created`).
- `GET /estimates`: paginated, `?status=` filter, role-scoped per the RBAC matrix (`client` reads `sent`-status estimates only, per design decision #3's "own estimate" scoping — everyone else with Estimation access reads all in-tenant estimates; `accountant` gets read access per the matrix).
- `GET /estimates/{id}`: 404 for cross-tenant/invisible (same `_get_estimate_or_404` pattern as every other entity), includes line items nested in the response (or a separate `estimate_line_items` list alongside — use judgment on response shape, but the frontend needs both the header fields and the current line items in one call for a usable Estimate-editing UI).
- Tests: create (both lead-scoped and project-scoped), create with neither/invalid association rejected, create against a Lead not in

`estimating+` status rejected, list, list-filtered, list-as-client-shows-sent-only, get, cross-tenant 404, non-admin/PM roles blocked on create.
- Verify: full suite green.
- Commit: `feat: add Estimate creation, listing, and detail routes`

Task 2.11: `PUT /estimates/{id}/lines`

Files: Modify `backend/app/routers/estimates.py`. Modify `backend/tests/test_estimates.py`.

- `require_role("admin", "project_manager")`. Batch-replaces ALL line items for the estimate in one call (per API spec: "Array of `{cost_catalog_item_id, quantity}`" — a full replace, not a partial patch/append).
- **409 if `estimate.is_snapshotted` is true** (design decision #4) — checked BEFORE any line item is touched, same "validate before mutating" discipline as every Phase 1 state-machine check.
- For each input line: look up the `cost_catalog_item_id` via `resolve_visible_catalog_items` (Task 2.4) — NOT a raw table query — since the item's *effective* `unit_rate` for this company must respect inheritance/override resolution (a PM building an estimate against a catalog item their branch has overridden must get the override's rate, not the ancestor's). Reject (422) a `cost_catalog_item_id` that doesn't resolve to any visible item for this company.
- `unit_rate_snapshot = resolved_item.unit_rate` at the moment of replacement (copied, not referenced — Inherited Invariant/schema doc Section 9's whole point). `line_total = quantity * unit_rate_snapshot` (Decimal arithmetic, never float — Inherited Invariant #9).
- This route does NOT itself recompute `estimate.subtotal/total` — that's `POST /estimates/{id}/calculate`'s job (Task 2.12), kept as a separate, explicit step per US-4.3 ("As a Project Manager, I can trigger a recalculation").
- Tests: replace with a fresh set of lines, replace clears out prior lines not in the new set (true replace, not append), snapshotted estimate rejects with 409 and applies NOTHING (verify via a subsequent `GET` that line items are unchanged), invalid `cost_catalog_item_id` rejected with 422, quantity/rate math uses Decimal (a test asserting `line_total` is exact, not float-rounded, for an input chosen specifically to expose float rounding error if it were present, e.g. `0.1 + 0.2`-style precision traps scaled to currency).
- Verify: full suite green.
- Commit: `feat: add batch line-item replacement with inheritance-aware rate resolution`

Task 2.12: `POST /estimates/{id}/calculate` — Calculation Engine

Files: Create `backend/app/services/estimate_calculation.py`. Modify `backend/app/routers/estimates.py`. Create `backend/tests/test_estimate_calculation.py`.

- Implements the fixed order from Technical Architecture Section 6, exactly: (1) line item base cost = `quantity × unit_rate` (already computed and stored per-line by Task 2.11 — this step re-reads, doesn't recompute, the stored `line_totals`), (2) category subtotals (group line items by their `cost_catalog_item`'s `category`, sum each group — used for the response's breakdown, not itself part of the running total calculation, unless the schema doc implies otherwise; if category subtotals are purely a display/reporting artifact, compute and return them without persisting a new column — don't add schema-doc scope creep for a value derivable on read), (3) overhead markup applied (`subtotal × (1 + markup_profile.overhead_pct / 100)`), (4) profit margin applied (`× (1 + markup_profile.profit_pct / 100)`), (5) tax liability calculated if applicable — **no tax rate/jurisdiction concept exists anywhere in the schema doc or functional requirements**; treat this as a no-op for Phase 2 (tax = 0) with an explicit code comment noting the gap (tax calculation needs a jurisdiction/rate model this schema doesn't define — do not invent one unrequested; flag it for a future phase's design pass instead of guessing at a rate table).
- All arithmetic uses `decimal.Decimal` exclusively — construct every intermediate value as `Decimal`, never mix in `float` at any step (a single `float(...)` call anywhere in this pipeline defeats the entire guarantee US-4.3's acceptance criterion requires).
- **409 if `estimate.is_snapshotted` is true** (design decision #4, same guard as Task 2.11).
- Writes `estimate.subtotal`/`estimate.total` (the persisted, authoritative values — per Technical Architecture Section 6's explicit instruction, "client-submitted totals are always ignored in favor of a server-side recompute").
- `require_role("admin", "project_manager")`.
- Tests: hand-computed expected values for a representative multi-line, multi-category estimate (per Test Strategy Section 3's explicit instruction — don't just assert "some non-null total," assert the EXACT expected number, computed by hand in the test itself); zero line items produces `subtotal=0, total=0` cleanly (no division-by-zero or null-arithmetic crash); a `Decimal`-precision-trap input (values that would visibly diverge under `float` arithmetic) produces the exact correct `Decimal` result; snapshotted estimate rejects with 409 and doesn't recompute.
- Verify: full suite green.

- Commit: `feat: add fixed-order Decimal calculation engine for Estimates`

Task 2.13: PDF Export — Template & Rendering Service

Files: Create `backend/app/services/pdf_export.py`, `backend/app/templates/estimate_pdf.html.jinja`. Modify `backend/pyproject.toml` (add `weasyprint`, `jinja2`). Create `backend/tests/test_estimate_pdf_export.py` (rendering-only tests; the async job wiring itself is Task 2.15).

- `render_estimate_pdf(estimate, line_items, markup_profile, company_name) -> bytes`: renders `estimate_pdf.html.jinja` (a clean, simple, company-name-labeled layout — line items table, category subtotals, overhead/profit/tax breakdown, final total; explicitly NOT attempting a logo or custom color scheme, per this plan's "Explicitly Out of Scope" section) via Jinja2, then converts the rendered HTML to PDF bytes via WeasyPrint. Pure function — no DB access, no filesystem writes, no async — this keeps it trivially testable and reusable from both a future synchronous code path and the Dramatiq worker (Task 2.15) without either needing to know about the other.
- All monetary values passed into the template are pre-formatted as strings in the calling code (e.g. `f"${value:,.2f}"`), not raw `Decimal`s handed to Jinja2 — keeps currency formatting logic in Python, testable, not scattered across template `{{ }}` expressions.
- Tests: rendering a representative estimate produces non-empty, valid PDF bytes (check the PDF magic-byte header `%PDF-`, not full visual/pixel verification — that's disproportionate for this test layer); rendering with zero line items doesn't crash; a company name and every line item's `category`/`name`/`quantity`/`unit_rate_snapshot`/`line_total` appear somewhere in the rendered HTML before PDF conversion (assert against the intermediate HTML string, which is far easier to inspect than PDF bytes — expose an internal `render_estimate_html()` helper `render_estimate_pdf()` calls, testable independently).
- Verify: full suite green.
- Commit: `feat: add Estimate PDF rendering service (WeasyPrint/Jinja2)`

Task 2.14: Async Job Infrastructure (Dramatiq + Redis)

Files: Create `backend/app/tasks/__init__.py`, `backend/app/tasks/broker.py`. Modify `backend/pyproject.toml` (add `dramatiq[redis]`), `docker-compose.yml` (new `worker` service, design decision #2).

- `broker.py`: configures a Dramatiq `RedisBroker` using `settings.redis_url` (already exists, unused until now), sets it as

Dramatiq's global default broker at module import time — this module must be imported before any `@dramatiq.actor`-decorated function is defined or invoked, so both `app/main.py` (for enqueuing from request handlers) and the worker's own entrypoint import it first.

- `docker-compose.yml`: add the `worker` service exactly as design decision #2 specifies (same image as `backend`, `command: dramatiq app.tasks.estimate_pdf`, `depends_on: {postgres: service_healthy, redis: service_healthy}`, `env_file: .env`, no published ports, `volumes: [./backend:/app]` matching `backend`'s own bind mount).
- No actor is defined yet in this task (that's Task 2.15) — this task only stands up the broker/infra plumbing and confirms it starts cleanly. Verify by hand: `docker compose up -d worker` brings up a healthy container that connects to Redis without crashing (check `docker compose logs worker` for a clean startup, not an error loop) — this is infra verification, not a pytest-covered behavior, note that explicitly in the task's own verification step rather than inventing a test that doesn't actually test anything meaningful.
- Run the full backend test suite (still green — nothing in the request path changed yet).
- Commit: `feat: add Dramatiq/Redis async job infrastructure`

Task 2.15: PDF Export — Async Job Wiring

Files: Create `backend/app/tasks/estimate_pdf.py`. Modify `backend/app/routers/estimates.py`. Modify `backend/tests/test_estimate_pdf_export.py`.

- `app/tasks/estimate_pdf.py`: a `@dramatiq.actor` function `generate_estimate_pdf(estimate_id: str)` (Dramatiq messages are JSON-serialized — pass `str(uuid)`, not a `uuid.UUID` object, and parse it back inside the actor). Opens its own `AsyncSession` (via `app/db.py`'s existing `SessionLocal`), calls `set_current_tenant()`/`set_current_user()` explicitly (Inherited Invariant #3's documented exception for worker code) using the estimate's own `company_id`/a system actor identity (there's no "requesting user" available inside a detached background job — use `actor_id=None` for the resulting audit log entry, or persist the requesting user's id in the enqueued message payload if attribution matters; **decide and document the choice explicitly**, don't leave it ambiguous), fetches the `Estimate` + its line items + markup profile + company name, calls `render_estimate_pdf()` (Task 2.13), writes the bytes to `{STORAGE_ROOT}/{company_id}/estimates/{estimate_id}.pdf` (reusing `document_storage.py`'s established path-safety conventions from Phase 1 — though the filename here is fully system-generated, not user-controlled, so path-traversal validation is inapplicable; still reuse the module's path-construction helpers for consistency rather than hand-

rolling a new path-join), updates `estimate.pdf_status='ready'`, `pdf_storage_path=<relative path>`, `pdf_generated_at=now()`, commits. On any exception during rendering/writing, updates `pdf_status='failed'` in a SEPARATE, fresh transaction (the failure handling itself must not be lost if the original transaction is what's rolling back) and re-raises so Dramatiq's own retry/dead-letter handling applies.

- `POST /estimates/{id}/export` (`estimates.py`): `require_role("admin", "project_manager")`. Validates the estimate exists/is visible. Sets `estimate.pdf_status='pending'`, commits (via the normal request-scoped session — this part IS in the request/response cycle and follows Inherited Invariant #3 normally), enqueues `generate_estimate_pdf.send(str(estimate.id))`, returns `202 Accepted` with the current (now-pending) `EstimateResponse` body. The actual PDF generation happens entirely out-of-band afterward.
- Tests (extending `test_estimate_pdf_export.py`): `POST /estimates/{id}/export` returns 202 and sets `pdf_status='pending'` immediately (synchronously verifiable within the same test, before the worker even runs); a DIRECT test of the `generate_estimate_pdf` actor function itself (call it as a plain async function in the test, not through the full Dramatiq broker/worker round-trip — that would require actually running a live worker process in the test suite, disproportionate for this layer) confirms it writes a real file to `STORAGE_ROOT` and updates `pdf_status='ready'`/`pdf_storage_path`/`pdf_generated_at` correctly; a forced-failure test (e.g., mock `render_estimate_pdf` to raise) confirms `pdf_status` lands on `'failed'`, not stuck on `'pending'` forever.
- Verify: full suite green.
- Commit: `feat: wire async PDF export job to POST /estimates/{id}/export`

Task 2.16: Estimate Tenant-Isolation Regression Tests

Files: Extend `backend/tests/test_tenant_isolation_phase2.py`.

- Same rigor as Task 2.6, applied to `estimates`/`estimate_line_items`: cross-tenant 404, header-spoofing blocked, one RLS-disable/re-enable proof for `estimates` specifically (representative of the plain, non-inherited policy shape both tables share — Inherited Invariant, same reasoning Phase 1 used to justify proving the mechanism once per policy-shape rather than per-table).
- Verify: full suite green, run twice.
- Commit: `test: add tenant-isolation regression coverage for Estimates`

Task 2.17: Esignature Model + Migration

Files: Create `backend/app/models/esignature.py`. Create `backend/migrations/versions/0006_esignatures_schema.py` (see Task 2.8's note — this must land BEFORE the estimates migration in the actual sequence; renumber if this plan's own numbering and the file-creation order drift, and note the correction in the commit).

- `Esignature`: `id`, `company_id` (FK), `signer_name`, `signer_email`, `signed_at` (timestamptz, not null), `ip_address` (Postgres `INET` type — SQLAlchemy's `sqlalchemy.dialects.postgresql.INET`), `signature_artifact_path` (text, not null), `document_type` (string, `CHECK` constrained `'estimate'/'change_order'`).
- Migration: `down_revision = "0005"`. Standard `tenant_isolation` policy. **`REVOKE UPDATE, DELETE ON esignatures FROM app_user`** — this is the one genuinely unconditionally-immutable table in this phase (Security & Compliance Section 7: "Indefinite / immutable… never deleted, even if the underlying Project or company is later deactivated"), so it DOES get Phase 1's blanket grant-level treatment (unlike `estimates`/`estimate_line_items` — design decision #4's conditional-immutability reasoning doesn't apply here, `esignatures` rows are immutable from the moment they're written, full stop).
- Verify by hand against live Postgres (RLS + policy + the REVOKE, same discipline as every prior migration task).
- Run the full suite (still green).
- Commit: `feat: add Esignature model and migration`

Task 2.18: Shared E-Signature Capture Service

Files: Create `backend/app/schemas/esignature.py`, `backend/app/services/esignature.py`. Create `backend/tests/test_esignatures.py`.

- `EsignatureCaptureRequest`: `signer_name`, `signer_email` (`EmailStr`) — `signed_at`/`ip_address` are NEVER client-supplied (server captures the real request timestamp and the actual connecting IP, per Security & Compliance Section 6's ESIGN Act intent-to-sign requirements — trusting a client-submitted IP/timestamp would defeat the entire evidentiary purpose of capturing them). `EsignatureResponse`: full model.
- `capture_esignature(session, *, company_id, signer_name, signer_email, ip_address, document_type, signature_artifact_bytes) -> Esignature`: the ONE shared function both Estimate approval (Task 2.19) and Change Order approval (Task 2.22) call — design decision #3/#6's "reuses the e-signature capability" made concrete as actual shared code, not two independent implementations that happen to look similar. Writes the signature

artifact to `{STORAGE_ROOT}/{company_id}/esignatures/{new_esignature_id}.<ext>` (reusing `document_storage.py` path helpers, same as Task 2.15's PDF path), inserts the `Esignature` row, returns it. Does NOT commit — caller (the approval endpoint) owns the transaction, same Inherited Invariant #3 as every other service function in this codebase.

- Extracting the real request IP: FastAPI's `Request.client.host` (needs the endpoint to accept a `Request` parameter) — note this can be a reverse-proxy's IP rather than the true origin if `X-Forwarded-For` isn't handled; this codebase's deployment target (Technical Architecture Section 8) sits behind Traefik/Nginx, so add a comment flagging that `X-Forwarded-For`-aware IP extraction is a real production-correctness concern deferred to whenever this proxy layer is actually configured (self-hosted deployment is a manual step outside this plan's scope, per the top-of-doc caveat) — don't silently ship IP capture that's wrong for the actual deployment topology without at least documenting the gap.
- `GET /esignatures/{id}` (API Specification Section 5: "Retrieve signature record (audit)"): add this route in this same task, alongside the capture service — role scoping per the RBAC matrix's Estimation row (Admin/PM full, Accountant read, Client can read their own signed records). 404 for cross-tenant/invisible, same `_get_x_or_404` pattern as everywhere else in this codebase. This is the one standalone read route this task needs; the capture service itself is invoked from Tasks 2.19/2.22, not exposed as its own route.
- Tests: capture produces a correct, immutable `Esignature` row with real (not client-supplied) `signed_at`/`ip_address`; the signature artifact file exists on disk afterward; a raw `UPDATE`/`DELETE` against `esignatures` as `app_user` is rejected (proving Task 2.17's `REVOKE`, same discipline as every prior immutability test); `GET /esignatures/{id}` returns the captured record, 404s cross-tenant.
- Verify: full suite green.
- Commit: `feat: add shared e-signature capture service and GET /esignatures/{id}`

Task 2.19: `POST /estimates/{id}/send-for-signature`, Approval, and Snapshotting

Files: Modify `backend/app/routers/estimates.py`. Modify `backend/tests/test_estimates.py`. Create `backend/tests/test_estimate_snapshotting.py`.

- `POST /estimates/{id}/send-for-signature`: `require_role("admin", "project_manager")`. Requires `estimate.total` to be set (i.e., `calculate` has run at least once — reject with 409 if `total IS NULL`, since sending an un-calculated estimate for approval makes no sense). Sets

`status="sent"`. No e-signature captured yet — this just marks it as awaiting the client's action.

- **The approval action itself** (US-4.5: "As a Client, I can review an emailed Estimate and approve it with an e-signature, or reject it with a reason") needs its own route not explicitly named in the API spec's literal table (which only lists `/estimates/{id}/send-for-signature` and a separate `/esignatures/{id}` GET) — same "spec is conceptual, not exhaustive" reasoning Phase 1 repeatedly applied (design decisions #3/#8 there). Add `POST /estimates/{id}/approve` and `POST /estimates/{id}/reject`, both `require_role("client")` (design decision #3's authenticated-in-app-client model), only legal from `status="sent"` (409 otherwise).
- `approve`: calls `capture_esignature()` (Task 2.18) with `document_type="estimate"`, sets `estimate.esignature_id`, `estimate.status="approved"`, **`estimate.is_snapshotted=true`** (design decision #4 — from this instant forward, `PUT /estimates/{id}/lines` and `POST /estimates/{id}/calculate` both 409 on this estimate permanently). Writes an audit log entry (`estimate.approved`). Publishes `ESTIMATE_APPROVED` (design decision #8) with `{estimate_id, project_id, company_id, approved_total}` — `project_id` may be `NULL` if the estimate was created against a bare Lead with no Project yet; document this nullability explicitly since Phase 3's eventual consumer will need to handle it.
- `reject`: requires a `reason` in the request body (US-4.5 explicit: "or reject it with a reason"). Sets `status="rejected"`. No snapshotting, no e-signature captured (a rejection isn't a signed document) — audit log entry (`estimate.rejected`, metadata `{reason}`) only.
- **Historical immutability regression test (the one Test Strategy Section 3 explicitly calls out):** after an estimate is approved and snapshotted, change the underlying `cost_catalog_items.unit_rate` for an item that was used in one of the estimate's line items, then re-fetch the approved estimate and assert its `total`/`subtotal`/every `line_total`/`unit_rate_snapshot` are UNCHANGED — this is the single most important test in this task, proving the snapshot genuinely decouples from live catalog data, not just that a flag got set.
- Tests: send-for-signature requires prior calculation, approve captures a real e-signature and flips `is_snapshotted`, approve publishes `ESTIMATE_APPROVED` with correct payload (capture via a test handler registered against `events`, same pattern Phase 1's `test_lead_state_machine.py` used for `LEAD_WON`), reject requires a reason and does NOT snapshot, non-client role cannot approve/reject (403), the catalog-price-change-after-approval immutability test above, attempting `PUT /lines/calculate` on an already-approved estimate 409s.
- Verify: full suite green.

- Commit: `feat: add Estimate send-for-signature, approve/reject, and historical snapshotting`

Task 2.20: ChangeOrder Model + Migration (0008)

Files: Create `backend/app/models/change_order.py`. Create `backend/migrations/versions/0008_change_orders_schema.py`.

- `ChangeOrder`: `id`, `project_id` (FK, cascade), `company_id` (FK), `description` (text, not null), `cost_delta` (`Numeric(12,2)`, not null — per US-3.6, this can legitimately be positive or negative, no `CHECK` constraint restricting sign), `schedule_impact_days` (int, default 0), `status` (string, default `"pending"`, `CHECK 'pending'/'approved'/'rejected'`), `esignature_id` (FK, nullable — set only once approved, mirroring `Estimate.esignature_id`'s own nullable-until-approved pattern), `created_at`.
- Migration: `down_revision = "0007"` (after estimates, per the corrected sequence from Task 2.8's note). Standard `tenant_isolation` policy, guarded-cast, `WITH CHECK`. No `REVOKE` needed at creation — `change_orders.status` transitions via the normal application-layer state machine (Task 2.21), same shape as `estimates.status`, not unconditionally immutable.
- This is the table Phase 1 explicitly deferred (see that plan's top-of-doc "Explicitly Out of Scope" note) — now unblocked since `esignatures` exists as of Task 2.17.
- Verify by hand against live Postgres, same discipline as every prior migration.
- Run the full suite (still green).
- Commit: `feat: add ChangeOrder model and migration`

Task 2.21: ChangeOrder Schemas + `POST /projects/{id}/change-orders`

Files: Create `backend/app/schemas/change_order.py`. Modify `backend/app/routers/projects.py` (or create `backend/app/routers/change_orders.py` if `projects.py` is unwieldy — same "use judgment, note the choice" latitude Phase 1's Task 1.14 was given for `tasks.py`). Create `backend/tests/test_change_orders.py`.

- `ChangeOrderCreateRequest`: `description`, `cost_delta` (`Decimal`), `schedule_impact_days` (optional, default 0). `ChangeOrderResponse`: full model.
- `POST /projects/{id}/change-orders`: `require_role("admin", "project_manager")` per API spec Section 4. **Only legal against an `active` Project** — US-3.6: "create a Change Order against an ACTIVE Project" (not draft/pre_construction/suspended/completed/archived);

reject with 409 if the project's current status isn't `active`. `status="pending"` at creation.

- `GET /projects/{id}/change-orders`: paginated list (not explicitly in the API spec's literal table, but necessary for a PM to see a project's Change Order history — same "spec is conceptual" reasoning applied repeatedly through this and the Phase 1 plan).
- Tests: create against an active project succeeds, create against a non-active project rejected with 409 (test at least `draft` and `completed` as representative illegal source states), list, cross-tenant 404, non-admin/PM roles blocked.
- Verify: full suite green.
- Commit: `feat: add Change Order creation and listing, scoped to active Projects`

Task 2.22: `POST /change-orders/{id}/send-for-signature`, Approval

Files: Modify the Change Order router from Task 2.21. Modify `backend/tests/test_change_orders.py`.

- `POST /change-orders/{id}/send-for-signature`: `require_role("admin", "project_manager")`. Only legal from `status="pending"` (409 otherwise, mirroring Estimate's own guard).
- `POST /change-orders/{id}/approve` / `.../reject`: same shape as Estimate's Task 2.19 — `require_role("client")`, calls the SAME `capture_esignature()` (Task 2.18) with `document_type="change_order"` on approve, sets `change_order.esignature_id`, `status="approved"`; reject requires a `reason`, sets `status="rejected"`. Audit log entries (`change_order.approved`/`change_order.rejected`).
- No snapshotting/immutability flag needed here — `change_orders` has no editable line-item collection the way Estimates do; once created, `description`/`cost_delta`/`schedule_impact_days` are never PATCH-able by any route in this plan (no update route exists at all, matching the "immutability by omission" pattern Phase 1's Lead-deletion rule used), so there's nothing further to lock down on approval beyond the `status` transition itself.
- Tests: send-for-signature guard, approve captures a real shared e-signature (assert it's genuinely the same `capture_esignature()` code path — e.g. by asserting the resulting `Esignature.document_type == "change_order"` and that the immutability/`REVOKE` guarantee from Task 2.17 applies identically), reject requires a reason, non-client roles blocked on approve/reject.
- Verify: full suite green.
- Commit: `feat: add Change Order send-for-signature and approval, reusing shared e-signature capture`

Task 2.23: Project-Completion-Blocked-by-Open-Change-Orders

Files: Modify `backend/app/services/project_transitions.py`, `backend/app/routers/projects.py`. Create `backend/tests/test_project_completion_blocked_by_change_orders.py`.

- Implements design decision #7 / Phase 1's own explicit "do not forget" comment (`project_transitions.py` lines 39–52, quoted in full in this plan's design decision #7 above). Add a check in `update_project_status` (`projects.py`) — NOT inside `is_legal_transition()` itself, which stays pure transition-table data with no DB queries (Phase 1's Task 1.18 established exactly this "state-machine-table stays pure, side effects/business-rule checks live in the router" split for `LEAD_WON`, apply the same shape here) — that runs BEFORE the transition is applied, when-and-only-when the target status is `completed`: query for any `change_orders` row against this `project_id` with `status != 'approved'`; if any exist, reject with `409` (a distinct, more specific error message than the plain illegal-transition 409, e.g. "Cannot complete project: N Change Order(s) pending approval").
- Applies to BOTH `active → completed` and `suspended → completed` (design decision #7's explicit warning against keying the check off "coming from suspended" specifically).
- Remove the now-obsolete "do not forget" comment block from `project_transitions.py`'s module docstring once this lands — replace it with a short note that the check now lives in `projects.py`'s `update_project_status`, pointing there, so a future reader of the transition table doesn't go looking for enforcement logic in the wrong file.
- Tests: a project with zero change orders can complete normally (no regression on the common case), a project with a `pending` change order cannot complete (409, from both `active` and `suspended`), a project with only `approved`/`rejected` change orders CAN complete (a rejected Change Order doesn't block completion — only genuinely open/pending ones do, per the business rule's literal "non-approved" wording… **reconsider this**: the functional requirement says "open (non-approved)," which taken literally would also block on `rejected` ones, but a rejected Change Order is resolved/closed, not open — treat `rejected` as NOT blocking, only `pending` blocks; document this interpretation explicitly as a judgment call in the code comment, since "non-approved" is genuinely ambiguous between "pending only" and "pending or rejected").
- Verify: full suite green.
- Commit: `feat: block Project completion while Change Orders are pending approval`

Task 2.24: Estimation & E-Signature Tenant-Isolation Regression Tests

Files: Extend `backend/tests/test_tenant_isolation_phase2.py`.

- Same rigor as Tasks 2.6/2.16, applied to `esignatures` and `change_orders`: cross-tenant 404, header-spoofing blocked. One RLS-disable/re-enable proof, on `change_orders` (representative of the plain non-inherited policy shape shared by `esignatures/estimates/change_orders/markup_profiles` — only `cost_catalog_items`' bidirectional policy needed its own dedicated proof in Task 2.6, everything else in this phase shares the ordinary Phase 1-style policy shape and doesn't need a second deep proof, matching the "prove the mechanism once per distinct shape" precedent).
- Include the parent/child hierarchy visibility case for `change_orders/estimates` (Phase 1's Task 1.17 precedent: a parent-company admin can see a child branch's Estimates/Change Orders, siblings cannot see each other's) — Project Management-adjacent data continues to matter for hierarchical visibility the same way it did in Phase 1.
- Verify: full suite green, run twice.
- Commit: `test: add tenant-isolation regression coverage for Esignatures and Change Orders`

Task 2.25: Full-Stack E2E Extension

Files: Modify `scripts/e2e_smoke_test.py`.

- Extend the existing live-stack smoke test (already covers Phase 0's checks and Phase 1's Lead→Won→Project flow) with the Phase 2 exit criterion, over real HTTP against real containers: create a Markup Profile, create a Cost Catalog item, create an Estimate against a Project, add line items via `PUT /estimates/{id}/lines`, calculate, send for signature, approve as a `client`-role user (capturing a real e-signature), assert the Estimate's `is_snapshotted` is now `true` and its totals match the hand-verifiable expected calculation, assert `GET /esignatures/{id}` returns the captured signature record. Optionally (if the worker container is straightforward to bring up in this script's existing `docker compose up -d --build` procedure) also exercise `POST /estimates/{id}/export` and poll `GET /estimates/{id}` until `pdf_status == 'ready'`, asserting a real file exists at the returned `pdf_storage_path` inside the backend container — if bringing up and reliably waiting on the `worker` service inside this script adds significant flakiness/complexity disproportionate to what an E2E smoke test needs, it's acceptable to defer the PDF-export leg of this

test to the backend's own async-job unit test (Task 2.15) and note that decision explicitly rather than silently skipping coverage.

- Run against the full local stack per Phase 0/1's established procedure (`docker compose up -d --build`, confirm `alembic upgrade head`, run the script, `docker compose down` without `-v`) — this phase's stack additionally includes the new `worker` service; bring it up too if the PDF-export leg above is included.
- Commit: `test: extend E2E smoke test with Estimate-to-Approval flow (Phase 2 exit criterion)`

Task 2.26: Full Regression Pass + Plan Closeout

Files: None (verification only) — update this document's exit-criteria checklist below.

- Full `pytest -v` from `backend/`, run twice for stability.
- Full RLS regression suite (`test_tenant_isolation.py` + `test_rls_policy_regression.py` + `test_tenant_isolation_phase1.py` + `test_tenant_isolation_phase2.py`) — confirm no regression in Phase 0/1's own tables from anything Phase 2 touched.
- CI green on a real GitHub Actions run (same discipline as Phase 0's Task 17 / Phase 1's Task 1.20 — open a verification PR, confirm the run, close it without merging pending an explicit user decision to merge). **Note:** the CI workflow (`backend-ci.yml`) will need a `worker`-adjacent check or at minimum confirmation the new `dramatiq[redis]`/`weasyprint` dependencies install cleanly in the CI environment — WeasyPrint in particular has native system-library dependencies (Pango, Cairo, GDK-PixBuf) beyond a plain `pip install`; verify the CI runner's base image has them or add the necessary `apt-get install` step to the workflow, don't assume it "just works" without checking.
- Commit: `docs: close out Phase 2 exit criteria checklist`

Phase 2 Exit Criteria Checklist

- ☐ Cost Catalog + Markup Profiles, with parent/child override inheritance (bidirectional RLS, dedup-and-prefer-closest resolution) — Tasks 2.1–2.6
- ☐ Estimate creation, line items, server-side fixed-order Decimal calculation pipeline — Tasks 2.7–2.16
- ☐ PDF export as a real async job (Dramatiq/Redis worker) — Tasks 2.13–2.15
- ☐ E-signature capture flow for Estimate approval, shared with Change Orders — Tasks 2.17–2.19

- ☐ Change Order creation + e-signature approval on active Projects — Tasks 2.20–2.23
- ☐ Historical immutability / snapshotting on Estimate approval, proven against a live catalog-price change — Task 2.19
- ☐ `ESTIMATE_APPROVED` event published (consumed starting Phase 3) — Task 2.19
- ☐ Project completion blocked by open (pending) Change Orders — Task 2.23
- ☐ Tenant isolation proven on every new table, including the Cost Catalog's novel bidirectional policy — Tasks 2.6, 2.16, 2.24
- ☐ Full-stack E2E exit criterion (Estimate → calculate → approve → snapshot) verified over real HTTP against real containers — Task 2.25
- ☐ Full regression suite green in CI — Task 2.26